

HRManagement Teknoloji ve Mimari Kararları

Her teknoloji için: neden seçildi, alternatifi neydi, kullanılmıyaydı ne olurdu ve bedeli ne oldu.

Proje: İnsan Kaynakları Yönetim Uygulaması

Yığın: .NET 10 · ASP.NET Core Web API + MVC · Dapper · SQL Server

Mimari: Clean Architecture · CQRS (MediatR) · Repository Pattern

Tarih: 27 Temmuz 2026

Bu belge nasıl kullanılır: Her bölüm aynı altı soruyu cevaplar — *ne yaptık, hangi problemi çözüyor, alternatifleri neydi, kullanılmıyaydı ne olurdu, bedeli ne, hangi sorular gelir*. Savunmada en çok işe yarayacak iki bölüm: **Bölüm 0** (bir isteğin baştan sona yolculuğu) ve **Bölüm 10** (projenin bilinçli zayıf noktaları). Sondaki **Ek A** hızlı tekrar için tek cümlelik cevap kartlarıdır.

İçindekiler

- 0** **Projeye Kuşbakışı**
30 saniyede proje tanıtımı + bir isteğin baştan sona yolculuğu

- 1** **Clean Architecture ve Katman Ayrımı**
Neden 5 proje? Tek projede yazsan olmaz mıydı?

- 2** **CQRS ve MediatR**
Neden Service class değil de Mediator deseni?

- 3** **FluentValidation ve Pipeline Behavior**
Validation'ı neden handler'ın içine yazmadın?

- 4** **Dapper, SQL Server ve Repository**
Neden EF Core kullanmadın? ORM'siz nasıl idare ediyorsun?

- 5** **İki Host: API + WebUI, Refit, BaseResponse**
Neden iki ayrı uygulama? Neden paylaşılan Contracts projesi yok?

- 6** **Kimlik Doğrulama: JWT + Cookie**
Neden ikisi birden? Token'ı neden localStorage'a koymadın?

- 7** **Hata Yönetimi ve Cross-Cutting Concerns**
ProblemDetails standardı varken neden kendi zarfın?

- 8** **Test Stratejisi**
Neden sadece Application test ediliyor? Neden Moq yok?

- 9** **Domain Modeli ve İş Kurallarının Yeri**
Entity'lerin anemik — bu DDD değil, sorun değil mi?

- 10** **Zayıf Noktalar ve Bilinçli Ödünler**
"Bunu neden yapmadın?" sorusunun hazır cevabı

- A** **Ek A — Hızlı Cevap Kartları**
Tek cümlelik savunmalar, son tekrar için

- B** **Ek B — Terimler Sözlüğü**

Projeye Kuşbakışı

Savunmanın ilk 60 saniyesi burada geçer. "Projeyi anlat" ve "bir isteği baştan sona anlat" soruları neredeyse her zaman sorulur; ikisinin de cevabı hazır olmalı.

1. Otuz saniyede proje

Bir şirketin İK süreçlerini yöneten web uygulaması. Beş rol var: **Admin, İK, Yönetici, Çalışan, Stajyer**. Uygulama çalışan ve stajyer kayıtlarını, departman/birim organizasyon şemasını, iki aşamalı izin onay akışını, stajyer görev-not takibini ve rol bazlı ekranları kapsıyor.

Teknik olarak **Clean Architecture** ile beş projeye bölünmüş bir .NET 10 çözümü. Use-case'ler **CQRS** deseniyle MediatR üzerinden yürüyor, veri erişimi **Dapper** ile ham SQL. Arayüz ayrı bir ASP.NET Core MVC uygulaması ve backend'e yalnızca **HTTP** üzerinden, **Refit** ile bağlanıyor — aralarında proje referansı yok.

PROJENİN TEK CÜMLELİK TASARIM FELSEFESİ

İş kuralları merkezde ve teknolojiden bağımsız; yetki her zaman sunucuda; her kural tek bir yerde yaşar.

Bu belgedeki kararların neredeyse tamamı bu üç cümleden türüyor.

2. Teknoloji seçimleri — tek bakışta

Teknoloji	Nerede	Tek cümlelik gerekçe
.NET 10 / C#	Her yer	Kurumsal .NET ekosistemi; staj ortamının standart yığını.
ASP.NET Core Web API	API	İş mantığının tek sahibi; ileride mobil/başka istemciler de aynı ucu tüketebilsin.
ASP.NET Core MVC	WebUI	Sunucuda render edilen sayfalar; ayrı bir JS build zinciri ve SPA karmaşası olmadan tam işlevsel arayüz.
MediatR 12.5	Application	Her use-case tek sınıf; validation/logging gibi ortak işler tek boru hattında.
FluentValidation	Application	Koşullu ve çapraz alan kuralları okunur biçimde, test edilebilir ayrı sınıflarda.
Dapper	Infrastructure	SQL'i gizlemeyen, ne çalıştığı belli olan ince eşleyici — öğrenme projesinde SQL'i görünür tutmak bilinçli tercih.
SQL Server	Veritabanı	İlişkisel model bu domaine tam oturuyor (departman→çalışan→izin); .NET ile en olgun entegrasyon.
Refit	WebUI	API sözleşmesi C# arayüzü olarak yazılır, istemci kodunu kütüphane üretir; elle HttpClient/JSON kodu yok.
JWT + Cookie	API + WebUI	İki farklı güven sınırı, iki farklı mekanizma; token tarayıcıya hiç inmez.
BCrypt	Infrastructure	Parola için kasıtlı olarak <i>yavaş</i> ve salt'lı hash — brute-force'u ekonomik olmaktan çıkarır.
xUnit	Tests	İş kurallarını veritabanı olmadan, milisaniyeler içinde doğrulamak.

3. Bir isteğin baştan sona yolculuğu

"Çalışan izin talebi oluşturuyor" senaryosunun tam yolu. Bu diyagramı ezberlemek, mimariyle ilgili soruların yarısını tek başına cevaplar:

TARAYICI

| POST /LeaveRequests/Create (form gönderimi, cookie ile)



WebUI – LeaveRequestsController

- | 1. Cookie doğrulanır → kullanıcı kimliği belli olur
- | 2. ViewModel → API request modeline çevrilir
- | 3. ILeaveRequestApi.CreateAsync(...) ← Refit arayüzü



BearerTokenHandler (DelegatingHandler)

- | Cookie ticket'ının içindeki JWT okunur
- | "Authorization: Bearer <token>" başlığı eklenir



HTTP

API – Middleware boru hattı

- | UseExceptionHandler → UseBaseResponseStatusCodes
- | UseAuthentication "sen kimsin?" (JWT imzası doğrulanır)
- | UseAuthorization "yetkin var mı?" (fallback policy)



LeaveRequestsController.Create

- | Kullanıcı Id'si JWT claim'inden okunur – gövdeden ASLA
- | CreateLeaveRequestCommand üretilir → _mediator.Send(command)



MediatR boru hattı

- | ValidationBehavior → CreateLeaveRequestCommandValidator
- | tarih sırası? enum geçerli? hastalık raporu var mı?
- | HATA → ValidationException → 400 + BaseResponse.Fail



CreateLeaveRequestCommandHandler ← İŞ KURALLARI

- | 1. Hesap hangi çalışana/stajyere ait?
- | 2. Kayıt aktif mi?
- | 3. Tarih başka talebiyle çakışıyor mu?
- | 4. Yıllık izin bakiyesi yetiyor mu?
- | 5. Hastalık izni ise → yönetici aşamasını atla, İK'ya düşür



ILeaveRequestRepository.AddAsync ← arayüz (Application)



LeaveRequestRepository ← Dapper (Infrastructure)

- | INSERT INTO LeaveRequests ... SELECT SCOPE_IDENTITY()



SQL SERVER



← yeni Id → BaseResponse<int>.Success(id) → JSON → Refit



WebUI: IsSuccess mi? → Redirect / hata göster

BU DİYAGRAMDAN ÇIKARILACAK ÜÇ SAVUNMA CÜMLESİ

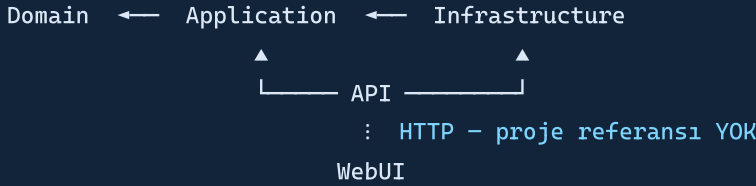
- **Yetki hiçbir zaman istemciye güvenmez.** Kullanıcı kimliği imzalı JWT claim'inden okunur; istek gövdesinden alınsaydı herkes "ben başkasıyım" diyebilirdi.
- **Input kontrolü ile iş kuralı ayrı yerlerde.** Veritabanına bakmayan kurallar Validator'da, bakanlar Handler'da.
- **Her katman bir sonrakini yalnızca arayüz üzerinden tanır.** Handler, Dapper'ı da SQL Server'ı da bilmez.

Clean Architecture ve Katman Ayrımı

Ana soru: Neden beş ayrı proje? Hepsini tek projede klasörlere ayırsan aynı şey olmaz mıydı?

1. Ne yaptık

Beş proje ve aralarında **tek yönlü** bir referans grafiği var:



Bunun kanıtı bir dokümanda değil, `.csproj` dosyalarında:

- `HRManagement.Domain.csproj` — **tek bir NuGet paketi bile yok**. Ne MediatR, ne Dapper, ne ASP.NET. Saf C#.
- `HRManagement.Application.csproj` — yalnızca Domain'e referans + MediatR/FluentValidation.
- `HRManagement.Infrastructure.csproj` — Dapper, SqlClient, BCrypt burada toplanmış.
- `HRManagement.WebUI.csproj` — **hiçbir iş katmanına referans vermiyor**, sadece Refit.

Asıl kritik nokta şu ters çevirme:

```
// Application/Interfaces/IEmployeeRepository.cs → arayüz burada TANIMLANIR
public interface IEmployeeRepository
{
    Task<Employee?> GetByUserIdAsync(int userId);
    ...
}

// Infrastructure/Persistence/EmployeeRepository.cs → burada UYGULANIR
public class EmployeeRepository : IEmployeeRepository { ... } // Dapper + SQL
```

DEPENDENCY INVERSION — SOLID'İN "D"Sİ, TEK CÜMLEDE

Arayüzün sahibi, onu kullanan taraftır — uygulayan taraf değil. Application "bana çalışan getirebilen bir şey lazım" der; o şeyin SQL mi, dosya mı, HTTP mi olduğunu bilmez. Bağımlılık oku böylece veritabanından *uzağa*, iş kurallarına *doğru* akar.

2. Hangi problemi çözüyor

Tek bir problem: **iş kurallarının, onları çalıştıran teknolojiden bağımsız yaşaması.**

Somut örnek — yıllık izin bakiyesi kuralı:

kullanılan + talep ≤ hak edilen + bir sonraki yılın hakkı

Bu bir **şirket politikası**; SQL Server'la, HTTP'yle, Razor'la hiçbir ilgisi yok. Aynı bir katmanda durduğu için bugün SQL Server'ı PostgreSQL yapsan, MVC'yi React'e çevirsen bu dosya **değişmez**. Kuralın ömrü, teknolojinin ömründen uzundur — mimarının bütün amacı bu asimetriyi kabul etmek.

3. Alternatifleri ve neden onlar değil

Alternatif	Nasıl olurdu	Neden seçilmedi
Tek proje, klasörlerle ayırım	Controllers/ , Services/ , Data/ klasörleri, hepsi tek .csproj	Klasör sınırını derleyici zorlamaz . Bir controller'ın içine <code>new SqlConnection(...)</code> yazmanı hiçbir şey engellemez. Yorgun bir cuma günü kural delinir ve kimse fark etmez.
Klasik N-Tier (UI → BLL → DAL)	DAL en altta, iş katmanı ona referans verir	Bağımlılık oku veritabanına doğru akar. İş katmanı DAL'ı, dolayısıyla EF/ADO.NET'i bilir. Clean Architecture'ın tek yapısal farkı bu oku ters çevirip merkeze iş kurallarını koyması.
Vertical Slice Architecture	Katman yok; her özellik kendi handler + SQL + endpoint'ini tek klasörde tutar	Meşru bir alternatif ve daha az boilerplate üretti. Ama katman disiplinini öğrenmek projenin amaçlarındandı; ayrıca paylaşılan kurallar (izin hakkı, görünürlük) birden çok dilime kopyalanma riski taşırdı.
Mikroservisler	İzin servisi, çalışan servisi, stajyer servisi ayrı ayrı	Tek ekip, tek veritabanı, ~10 tablo. Dağıtık sistemin bedeli (ağ hataları, dağıtık transaction, ayrı deploy hattı) kazancından kat kat büyük olurdu.

4. Kullanmasaydık ne olurdu

İzin bakiyesi kuralını controller'ın içine yazdığımızı düşünelim:

```
[HttpPost]
public async Task<IActionResult> Create(CreateLeaveRequest request)
{
    using var conn = new SqlConnection(_config.GetConnectionString("Default"));
    var used = await conn.QuerySingleAsync<int>("SELECT SUM(WorkingDays) ...");
    if (used + requested > accrued + nextGrant) return BadRequest(...);
    ...
}
```

Dört somut sonuç:

1. **Test etmek neredeyse imkânsız olur.** "Bir yılını doldurmamış çalışan yıllık izin alamaz" kuralını doğrulamak için ayakta bir SQL Server, seed verisi ve HTTP isteği gerekirdi. Şu an ilgili testler veritabanına hiç dokunmadan,

milisaniyelerde çalışıyor.

2. **Kural çoğalır ve ayrışır.** Aynı kontrol mobil API'de, toplu içe aktarma ekranında, admin panelinde yeniden yazılır. Üçü zamanla birbirinden ayrışır ve hangisinin doğru olduğu bilinmez hâle gelir.
3. **Kural görünmez olur.** "Şirketin izin politikası nedir?" sorusunun cevabı 400 satırlık controller'ların içine dağılır. Şimdi cevap tek yerde: [Application/Services/LeaveEntitlement.cs](#).
4. **Teknoloji değişimi = yeniden yazım.** Dapper'ı EF Core'a çevirmek, iş kurallarını da baştan yazmak demek olurdu.

MİMARİNİN İDDİASININ KANITI

[tests/HRManagement.Application.Tests.csproj](#) yalnızca **Application**'a referans veriyor — Infrastructure'a değil. Yani iş kuralları gerçekten veritabanı olmadan test edilebiliyor. Mimari "iyi olur" diye iddia edilen bir şey değil, çalışan bir test projesiyle ispatlanmış durumda.

5. Bedeli — dürüst kısım

BUNLARI SAVUNMADA KENDİN SÖYLERSEN GÜÇLÜ GÖRÜNÜRSÜN

- **Boilerplate.** "Departman ekle" gibi basit bir CRUD için 4 dosya (Command, Handler, Validator, Repository metodu). Tek katmanlı bir projede bu tek metot olurdu.
- **Aynı verinin çoklu temsili.** Bir çalışan verisi dört ayrı şekilde yaşıyor: [Domain/Entities/Employee](#) → [Application/dto/Employee](#) → [API/Models/Employees](#) → [WebUI/Models/Api/Employees](#). Bilinçli bir ödün (Bölüm 5), ama bedeli elle senkron tutmak.
- **Öğrenme eğrisi.** Projeye yeni katılan biri "bu kod nerede?" sorusunu cevaplamak için önce mimariyi öğrenmek zorunda.
- **Küçük projede israf olurdu.** 3 tablolü bir uygulamada bu yapı gereksiz ağırlık yaratırdı. Bu proje (10 tablo, 5 rol, iki aşamalı onay akışı, rol bazlı görünürlük kuralları) eşiği net biçimde geçiyor.

6. Gelmesi muhtemel sorular

Katman kuralını nasıl garanti ediyorsun? Doküman yazmak yetmez.

→ Doküman değil, **derleyici** garanti ediyor. WebUI'nin [.csproj](#) 'unda Application referansı yok; bir controller'da [using HRManagement.Application](#) yazsam proje derlenmez. Mimari kural, build hatası olarak somutlaşıyor.

Domain katmanı neden bomboş, sadece entity var?

→ "Boş" olması özellik. Hiçbir NuGet paketi almıyor — hiçbir framework'e bağımlı değil. Uygulamanın en uzun ömürlü, en az değişen parçası orası. (Anemik model eleştirisi ayrı ve gerçek bir tartışma; Bölüm 9'da ele alınıyor.)

Application, Infrastructure'ı hiç bilmiyorsa somut repository'yi kim veriyor?

→ **Composition root** — [API/Program.cs](#). Uygulamanın tek noktasında, çalışma anında `IEmployeeRepository` → `EmployeeRepository` eşlemesi yapılıyor. Derleme zamanında Application hâlâ hiçbir şey bilmiyor; bağlantı runtime'da DI konteyneri tarafından kuruluyor.

Bu Clean Architecture mi, Onion mu, Hexagonal mi?

→ Üçü de aynı ilkenin farklı isimleri: **bağımlılıklar içeri akar, iş kuralları merkezdedir**. Hexagonal "port ve adaptör", Onion "iç içe halkalar" metaforunu kullanır; bizimki isimlendirme olarak Clean Architecture.

Bu mimariyi savunmayacağın bir durum var mı?

→ Var. Tek ekranlı bir CRUD aracı, ömrü birkaç aylık bir iç araç ya da prototip yazıyorsam bu yapı sadece yavaşlatır. Clean Architecture'ın getirisi **zamanla** ödenir; ömrü kısa olan projede o zaman hiç gelmez.

CQRS ve MediatR

Ana soru: `IEmployeeService` gibi normal bir servis sınıfı yazsan olmaz mıydı? Araya mediator koymanın kazancı ne?

1. Ne yaptık

Her use-case, kendi klasöründe üç dosyadan oluşuyor:

```
Application/Features/LeaveRequests/Commands/CreateLeaveRequest/  
├─ CreateLeaveRequestCommand.cs           // istek - sealed record  
├─ CreateLeaveRequestCommandHandler.cs     // iş kuralları - sealed class  
└─ CreateLeaveRequestCommandValidator.cs   // input kontrolü
```

```
public sealed record CreateLeaveRequestCommand(  
    int RequesterUserId, LeaveType Type, DateTime StartDate,  
    DateTime EndDate, string? Description, string? MedicalReport) : IRequest<int>;  
  
public sealed class CreateLeaveRequestCommandHandler  
    : IRequestHandler<CreateLeaveRequestCommand, int> { ... }
```

Controller ise use-case'lerin hiçbirini tanıtmıyor; yalnızca `IMediator` 'ı tanıyor:

```
public LeaveRequestsController(IMediator mediator) { _mediator = mediator; }  
  
[HttpGet("pending-approvals")]  
public async Task<IActionResult> GetPendingApprovals()  
{  
    var items = await _mediator.Send(new GetPendingApprovalsQuery(CurrentUserId()));  
    ...  
}
```

Kayıt tek yerde ve otomatik:

```
services.AddMediatR(cfg =>  
{  
    cfg.RegisterServicesFromAssembly(assembly); // tüm handler'lar otomatik bulunur  
    cfg.AddOpenBehavior(typeof(ValidationBehavior<,>)); // ortak boru hattı halkası  
});
```

2. Hangi problemi çözüyor

a) Servis sınıflarının şişmesini engeller

Klasik yaklaşımda `LeaveRequestService` zamanla 800 satıra çıkar: oluştur, onayla, reddet, listele, detay, bekleyenler... Hepsi aynı sınıfta, hepsi aynı constructor'da yedi bağımlılık taşır. Burada her operasyon **kendi sınıfı**

— yalnızca ihtiyacı olan üç repository'yi enjekte ediyor. Tek Sorumluluk İlkesi (SOLID'in "S"si) sınıf seviyesinde gerçekten uygulanıyor.

b) Cross-cutting concern'leri tek yerde toplar

Asıl kazanç **pipeline behavior**. Her mesaj handler'a ulaşmadan önce `ValidationBehavior` 'dan geçiyor. Yarın "her komutu logla" ya da "her komutu transaction'a sar" istersek, 26 handler'a dokunmadan tek bir sınıf ekliyoruz.

```
Send(command)
|
|→ ValidationBehavior      ← şu an var
|   |→ (LoggingBehavior)   ← eklenebilir, handler'lar değişmez
|   |   |→ (TransactionBehavior)
|   |       |→ HANDLER ← asıl iş
```

c) CQRS: okuma ile yazmayı ayırır

Command sistemde değişiklik yapar, **Query** yan etkisizdir. Klasör yapısı bu ayrımı fiziksel hâle getiriyor. Faydası: bir sorunun "acaba bu bir şeyi değiştiriyor mu?" cevabı klasör adından okunuyor; ayrıca okuma tarafı entity'ye sadık kalmak zorunda değil, doğrudan ekranın istediği DTO'yu üretebiliyor.

DİKKAT — BU "TAM CQRS" DEĞİL, BİLEREK

Literatürdeki tam CQRS'te okuma ve yazma **ayrı veritabanları** kullanır ve aralarında event'lerle senkronizasyon olur. Bizde tek veritabanı var; ayırım yalnızca **kod düzeyinde**. "CQRS kullandın mı?" sorusuna doğru cevap: "*Kod seviyesinde evet, ayrı okuma modeli/veritabanı anlamında hayır — bu ölçekte eventual consistency'nin bedeli gereksizdi.*" Bu ayrımı bilmek, ezber cevaptan ayırt edilmeni sağlar.

3. Alternatifleri ve neden onlar değil

Alternatif	Artısı	Neden bu proje için değil
Servis sınıfı <code>IEmployeeService</code>	Basit, dolaylılık yok; "Go to definition" doğrudan çalışır; yeni gelen anında anlar.	Cross-cutting'i her metoda elle eklemek gerekir. Sınıflar zamanla şişer. Controller 5-6 servis enjekte etmeye başlar.
Kendi mediator'ını yazmak	~50 satır; dış bağımlılık ve lisans riski yok.	Behavior zincirini, generic çözümlemeyi ve <code>void / Notification</code> senaryolarını doğru yazmak görüldüğünden zor; hata payı yüksek.
Wolverine / Brighter	MediatR'ın açık kaynak alternatifleri; mesajlaşma/outbox gibi ek yetenekler.	Ekosistem tanınırlığı ve dokümantasyon MediatR kadar geniş değil; öğrenme projesinde en yaygın olanı öğrenmek daha değerli.
Minimal API + endpoint filter	Daha az dosya; ASP.NET Core'un yerleşik filtreleriyle cross-cutting mümkün.	Cross-cutting yalnızca HTTP sınırında çalışır — iş katmanı çağrıldığında (ör. arka plan işi) kurallar devrede olmaz.

4. Kullanmasaydık ne olurdu

En somut kayıp `ValidationBehavior`. Onsuz, 26 handler'ın her birinin ilk satırlarında şu blok tekrar ederdi:

```
var result = await _validator.ValidateAsync(request, cancellationTokens);  
if (!result.IsValid) throw new ValidationException(result.Errors);
```

Bunu **bir handler'da yazmayı unutmak** derleme hatası vermez, test de kolayca yakalamaz — sadece o uç sessizce doğrulamasız kalır. Yani unutulan bir satır, doğrudan bir güvenlik/veri bütünlüğü açığına dönüşür. Behavior yaklaşımında unutmak **mümkün değil**: kural boru hattında, handler'ın haberi bile yok.

5. Bedeli

DÜRÜST MALİYET LİSTESİ

- IDE navigasyonu zayıflar.** `_mediator.Send(new XCommand())` satırında "Go to definition" seni handler'a götürmez — eşleme çalışma anında yapılır. Handler'ı isimle aramak gerekir.
- Dolaylılık.** Yeni gelen biri "bu kod nereden çağrılıyor?" sorusunu sorar. Küçük ekipte bu gerçek bir sürtünme.
- Dosya sayısı patlar.** Her use-case için 3 dosya; projede onlarca klasör.
- Performans (ihmal edilebilir ama var).** Her istekte reflection tabanlı handler çözümlemesi ve behavior zinciri. Web uygulaması ölçeğinde ölçülemez, ama "bedeli sıfır" demek doğru olmaz.

LİSANS RİSKİ — BU CEVABI MUTLAKA HAZIR TUT

MediatR **12.5.0** sürümünde sabitlenmiş durumda. Sebebi teknik değil, hukuki: kütüphane **13. sürümden itibaren ticari lisansa** geçti. Projede sürüm bilinçli olarak dondurulmuş ve bu karar [CLAUDE.md](#)'ye yazılmış.

Buradan çıkan genel ders: bir bağımlılık seçerken yalnızca bugünkü yeteneklerine değil, **lisans geleceğine ve bakım modeline** de bakmak gerekir. Savunmada bunu kendiliğinden söylemek, teknoloji seçimini sadece "popüler olduğu için" yapmadığını gösterir.

6. Gelmesi muhtemel sorular

Mediator deseni tam olarak nedir?

→ Birbiriyle konuşması gereken nesnelerin **doğrudan birbirini tanımak yerine** ortak bir aracı üzerinden haberleşmesi. Burada controller, hangi sınıfın isteği karşılayacağını bilmez; sadece "şu mesajı gönder" der. Bağlantı sayısı $N \times N$ 'den N 'e düşer.

Command neden `record`, handler neden `class`?

→ Mesaj bir kez oluşturulup boru hattı boyunca **değişmeden** taşınır — `record` değişmezliği ve değer eşitliğini ücretsiz verir (testlerde `Assert.Equal` doğrudan çalışır, otomatik `ToString()` loglamada işe yarar). Handler ise bağımlılık taşıyan, davranışı olan bir nesne; orada `class` doğru araç.

`CancellationToken` 'ı neden her yere taşıyorsun?

→ Kullanıcı sayfayı kapattığında ya da istek zaman aşımına uğradığında, arkada çalışan sorgunun devam etmesinin anlamı yok. Token repository'ye kadar iletilirse veritabanı çağırısı da iptal edilebilir. Hiçbir katmanda yutulmaması bilinçli bir kural.

MediatR olmasaydı aynı faydayı nasıl alırdın?

→ DI konteynerinin **decorator** desteğiyle. `IEmployeeService` 'i saran bir `ValidatingEmployeeService` yazılabilir. Aynı sonucu verir ama her servis arayüzü için ayrı decorator yazmak gerekir; MediatR'da tek generic behavior tüm mesajları kapsar.

FluentValidation ve Pipeline Behavior

Ana soru: Validation'ı neden handler'ın içine yazmadın? Ve zaten .NET'te `[Required]` attribute'ları varken neden ayrı bir kütüphane?

1. Ne yaptık

Her komutun yanında bir validator sınıfı duruyor ve boru hattı onu otomatik çalıştırıyor:

```
public sealed class CreateLeaveRequestCommandValidator
    : AbstractValidator<CreateLeaveRequestCommand>
{
    public CreateLeaveRequestCommandValidator()
    {
        RuleFor(c => c.Type)
            .IsInEnum().WithMessage("Geçerli bir izin türü seçilmelidir.");

        // Çapraz alan kuralı: iki alan birbirine göre denetleniyor
        RuleFor(c => c.EndDate)
            .GreaterThanOrEqualTo(c => c.StartDate)
            .WithMessage("Başlangıç tarihi bitiş tarihinden sonra olamaz.");

        // KOŞULLU kural: yalnızca hastalık izninde rapor zorunlu
        RuleFor(c => c.MedicalReport)
            .NotEmpty().When(c => c.Type == LeaveType.Sick)
            .WithMessage("Hastalık izni için rapor bilgisi zorunludur.");
    }
}
```

2. Hangi problemi çözüyor — asıl mesele "ayrım"

Bu bölümün en değerli cevabı validation kütüphanesinin kendisi değil, **iki tür kuralın birbirinden ayrılması**:

	Input Validation → Validator	İş Kuralı → Handler
Tanımı	Gelen veri <i>kendi içinde</i> geçerli mi?	Bu işlem, sistemin <i>mevcut durumuna</i> göre yapılabilir mi?
Ayırt edici test	Veritabanına bakmadan cevaplanabilir	Cevaplamak için veritabanına bakmak gerekir
Projeden örnek	Bitiş ≥ başlangıç; izin türü geçerli bir enum; açıklama ≤ 500 karakter; hastalık izninde rapor dolu	Yıllık izin bakiyesi yetiyor mu; tarihler başka bir talebiyle çakışıyor mu; çalışan kaydı aktif mi; staj süresi dolmuş mu
Değişme sıklığı	Nadir — form değişirse değişir	Sık — şirket politikası değişirse değişir

SAVUNMADA ALTIN CÜMLE

"Validation'ı nereye koyarsın?" sorusunun en iyi cevabı "Validator'a" değil, şudur: **"İkiye ayırırım. Veritabanına bakmadan cevaplanabilen her şey input validation'dır ve boru hattında çalışır; sistemin durumuna bakan her şey iş kuralıdır ve handler'da kalır. İkisini karıştırmak, iş kurallarını framework'e bağımlı hâle getirir."**

3. Neden DataAnnotations değil

Konu	DataAnnotations	FluentValidation
Koşullu kural "Sadece hastalık izninde zorunlu"	Attribute ile ifade edilemez; özel attribute yazmak gerekir	<code>.When(...)</code> ile tek satır
Çapraz alan kuralı "Bitiş ≥ başlangıç"	<code>IValidatableObject</code> implement etmek gerekir — model kirlenir	<code>GreaterThanOrEqualTo(c => c.StartDate)</code>
Kuralın yeri	Modelin üstünde — mesaj sınıfı validation framework'üne bağlanır	Ayrı sınıfta — mesaj temiz kalır
Test edilebilirlik	Attribute'ları izole test etmek zahmetli	Validator ayrı bir sınıf; doğrudan <code>new</code> 'lenip test edilir
Hata mesajı kontrolü	Sınırlı, kültüre bağlı sürprizler	Her kural için açık <code>.WithMessage(...)</code>

Projede validator'lar için ayrı testler var ([Validators/CreateLeaveRequestCommandValidatorTests.cs](#)) — yukarıdaki "test edilebilirlik" iddiasının somut karşılığı bu.

4. Kullanmasaydık ne olurdu

Handler şöyle görünürdü — input kontrolü ile iş kuralı iç içe geçmiş hâlde:

```
public async Task<int> Handle(CreateLeaveRequestCommand r, CancellationToken ct)
{
    if (r.EndDate < r.StartDate) throw new Exception("Tarih hatalı");
    if (!Enum.IsDefined(r.Type)) throw new Exception("Tür hatalı");
    if (r.Description?.Length > 500) throw new Exception("Çok uzun");
    if (r.Type == LeaveType.Sick && string.IsNullOrEmpty(r.MedicalReport)) ...
    // ... 15 satır daha, sonra nihayet asıl iş başlıyor
}
```

Üç sonuç:

- **Asıl iş kuralları gürültü altında kaybolur.** Handler'ı okuyan kişi, izin politikasını görmek için 15 satır kontrol kodunu atlamak zorunda kalır.
- **Hata mesajları tutarsızlaşır.** Her handler kendi mesaj biçimini uydurur; istemci tek tip parse edemez.

- **Tek tek hata yerine ilk hatada durulur.** FluentValidation tüm alanları toplayıp *hepsini birden* döndürebilir; elle yazılan `if` zinciri ilk hatada kesilir ve kullanıcı formu 4 kez göndermek zorunda kalır.

5. Bedeli

DÜRÜST MALİYET

- **Ekstra bağımlılık ve dosya.** Her komut için bir validator sınıfı daha.
- **Kural iki yerde yaşıyor.** WebUI tarafında da form doğrulaması var (UX için — kullanıcı sunucuya gitmeden hatayı görsün). Bu kasıtlı bir tekrar, ama senkron tutma yükü gerçek. **Otorite her zaman sunucudur;** istemci tarafı yalnızca kullanıcı deneyimi içindir.
- **Sessiz atlama riski.** Bir komut için validator yazmayı unutursan boru hattı sessizce geçer (`if (!_validators.Any()) return await next(...)`). Bu bilinçli bir tasarım — basit query'ler validator istemez — ama "yazmayı unuttum" ile "gerekmiyor" durumları dışarıdan aynı görünür.

6. Açık tartışma: Exception mı, Result mı?

Projede iş kuralı reddi **exception fırlatarak** bildiriliyor (`DataAnnotations.ValidationException`), global handler bunu 400'e çeviriyor. Proje rehberi ([CLAUDE.md](#)) ise `Result` deseni öngörüyor — kodda böyle bir tip **yok**. Mentörün bu farkı sorması ihtimaline karşı iki tarafı da bilmek gerekiyor:

Exception yaklaşımı (mevcut)	Result yaklaşımı (rehberdeki hedef)
+ Handler kodu akıcı kalır; hata yolu iç içe <code>if</code> üretmez + Tek noktada (global handler) tutarlı biçimde 400'e çevrilir – "Beklenen" bir durum için exception kullanmak tartışmalıdır – Derleyici seni hatayı ele almaya <i>zorlamaz</i>; unutulabilir – Exception fırlatmanın ölçülebilir bir maliyeti vardır	+ Hata, dönüş tipinin parçasıdır — derleyici ele almaya zorlar + "Kural reddi" ile "beklenmeyen çökme" tip düzeyinde ayrışır + Performans maliyeti yok – Her katmanda <code>Result</code> 'ı taşımak ve açmak boilerplate üretir – Boru hattı ve controller tarafında ek eşleme kodu gerekir

Savunmada söylenecek dürüst cümle: *"Şu an exception kullanıyoruz; global handler sayesinde dışarıya tutarlı bir sözleşme veriyor. Result'a geçmek daha doğru olurdu çünkü hata yolunu derleyici denetlerdi — projenin bilinen teknik borçlarından biri."*

7. Gelmesi muhtemel sorular

Validation'ı istemcide yapsan yetmez mi?

→ Asla. İstemci tarafı doğrulama yalnızca **kullanıcı deneyimi** içindir; tarayıcı geliştirici konsolundan ya da doğrudan HTTP isteğiyle atlatılabilir. Otorite her zaman sunucudadır. Aynı ilke rol bazlı menü gizlemede de geçerli: menüyü gizlemek yetki değildir, yetki API'dedir.

Validator hangi anda çalışıyor?

→ Handler'dan **önce**, MediatR boru hattında. Hata varsa handler hiç çağrılmaz — yani geçersiz veri iş kurallarına hiç ulaşmaz.

Aynı kural hem validator'da hem handler'da olabilir mi?

→ Projede bir örneği var: hastalık izninde rapor zorunluluğu her ikisinde de kontrol ediliyor. Sebep, handler'ın **kendi başına da doğru davranması**: boru hattı devre dışıyken (ör. doğrudan birim testinde) kural yine de korunur. Savunulabilir bir kemer-askı yaklaşımı, ama bilinçli olduğunu belirtmek gerekir.

Dapper, SQL Server ve Repository Pattern

Ana soru: .NET dünyasının standardı EF Core iken neden Dapper? Migration'sız nasıl yaşıyorsun?

1. Ne yaptık

Veri erişimi ham SQL + Dapper ile, tek bir bağlantı fabrikası üzerinden:

```
public class LeaveRequestRepository : ILeaveRequestRepository
{
    public async Task<int> AddAsync(LeaveRequest leaveRequest)
    {
        const string sql = @"
            INSERT INTO LeaveRequests (EmployeeId, InternId, Type, StartDate, ...)
            VALUES (@EmployeeId, @InternId, @Type, @StartDate, ...);
            SELECT CAST(SCOPE_IDENTITY() AS INT);";

        using var connection = _connectionFactory.CreateConnection();
        return await connection.QuerySingleAsync<int>(sql, leaveRequest);
    }
}
```

Şema değişiklikleri [db/](#) klasöründe numaralı, **idempotent** (tekrar çalıştırılabilir) SQL script'leriyle takip ediliyor: `01_schema.sql ... 15_backfill_employee_gender.sql`.

2. Dapper ne yapar, ne yapmaz

Dapper bir **micro-ORM**: SQL'i sen yazarsın, o yalnızca sonucu C# nesnesine eşler ve parametreleri güvenli biçimde bağlar. EF Core ise **full ORM**: SQL'i senin yerine üretir, değişiklikleri takip eder, ilişkileri yönetir.

Yetenek	Dapper (bizim seçim)	EF Core
SQL üretimi	Sen yazarsın	LINQ'tan otomatik üretilir
Change tracking	Yok	Var
Migration	Yok — elle script	<code>dotnet ef migrations</code>
İlişki gezinme	Elle JOIN / ayrı sorgu	<code>Include()</code>
SQL injection koruması	Var (parametrelili sorgu)	Var
Performans	ADO.NET'e çok yakın	Genelde daha yavaş (takip + çeviri maliyeti)
Öğrenme süresi	~1 saat	Haftalar (davranışını gerçekten bilmek için)

3. Neden Dapper seçildi

1. **SQL'i görünür tutmak — öğrenme amacı.** Bu bir staj projesi; ORM'in ürettiği sorguyu okumak yerine sorguyu doğrudan yazmak, ilişkisel düşünmeyi öğretir. EF Core ile başlayan çoğu geliştirici, ürettiği SQL'i hiç görmeden yıllar geçirir ve performans sorununda çaresiz kalır.
2. **Sürpriz yok.** Veritabanına giden sorgu, dosyada yazan sorgunun aynısı. EF Core'da "neden bu sorgu 4 saniye sürdü" sorusu ancak profiler ile cevaplanır.
3. **N+1 problemi kazara oluşamaz.** EF Core'da lazy loading açıkken bir döngü içinde `employee.Department.Name` yazmak, 500 çalışan için 500 ek sorgu üretir ve bunu **kod okunarak fark etmek zordur**. Dapper'da o sorguyu sen yazmak zorunda olduğun için problem görünür hâle gelir.
4. **Karmaşık okuma sorguları doğal.** Dashboard ve detay ekranları birden çok tabloyu birleştiren, ekrana özel şekiller döndürüyor. Bunlar LINQ'ta zorlanır, SQL'de tek sorgudur.

4. Kullanmasaydık — EF Core seçseydik ne kazanır, ne kaybederdik

KAZANIRDIK

- **Migration altyapısı.** Şu an `db/` klasöründe elle yazılan idempotent script'ler var ve "veritabanı hangi sürümde?" sorusunun otomatik cevabı yok.
- **Daha az tekrar kod.** `UpdateAsync` metodunda 12 kolon elle yazılıyor; yeni bir alan eklendiğinde INSERT, UPDATE ve SELECT'i ayrı ayrı güncellemek gerekiyor — unutmak sessiz hata üretir.
- **Unit of Work ve transaction** hazır gelirdi (Bölüm 10'daki eksiklerden biri).
- **Concurrency token** desteği (aynı kaydı iki kişinin aynı anda güncellemesi).

KAYBEDERDİK

- SQL görünürlüğü ve öğrenme değeri.
- Öngörülebilir performans; EF'in davranışını öğrenmek için ayrı bir öğrenme yükü.
- Domain entity'lerinin saflığı — EF genelde entity'lere navigation property ve konfigürasyon getirir, Domain katmanı EF'e bulaşmaya başlar.

5. Repository Pattern — neden bu projede gerçekten değerli

Repository, bugünlerde sık eleştirilen bir desendir. Eleştirinin özü şudur: "EF Core'un `DbContext` 'i zaten Repository + Unit of Work'tür; üstüne bir katman daha koymak gereksiz soyutlamadır." Bu eleştirisi **EF Core kullanılan projeler için haklıdır**.

Ama bizim durumumuzda tersi geçerli: Dapper bir soyutlama sunmaz, ham SQL'dir. Repository olmasaydı SQL metinleri **Application katmanının içine sızardı** ve Bölüm 1'de kurduğumuz her şey çökerdi. Yani burada repository, mimarinin taşıyıcı kolonlarından biri.

İkinci ve ölçülebilir fayda: **test edilebilirlik**.

```
// tests/.../ApproveLeaveRequestCommandHandlerTests.cs
var handler = new ApproveLeaveRequestCommandHandler(
    new FakeLeaveRequestRepository(leaveRequest), // ← veritabanı yok
    new FakeEmployeeRepository(ownerEmployee),
    new FakeUserRepository(users),
    new LeaveApprovalGuard(...));

await handler.Handle(new ApproveLeaveRequestCommand(42, AdminUserId), CancellationToken.None);

Assert.Equal(LeaveStatus.Approved, repo.Updated!.Status);
```

Repository arayüzü olmasaydı bu test için ayakta bir SQL Server ve hazırlanmış veri gerekirdi — yani pratikte hiç yazılmazdı.

6. Bedeli

DAPPER'IN BU PROJEDE YARATTIĞI SOMUT BORÇLAR

- **SELECT * kullanımı.** Bazı sorgular tüm kolonları çekiyor. Kolon eklendiğinde/adı değiştiğinde kod sessizce kırılabilir ve gereksiz veri taşınır. Doğrusu kolonları açıkça listelemek.
- **Şema değişikliği manuel iş çıkarıyor.** Yeni bir alan: entity + INSERT + UPDATE + SELECT + SQL script. Beş dokunuş, hepsi elle.
- **Transaction yok.** Her repository metodu kendi bağlantısını açıyor. Çok adımlı bir işlemde (ör. hesap talebi onayı: kullanıcı oluştur + çalışan kaydını bağla) ortada hata olursa **yarım kalmış durum** oluşabilir. Bilinen ve kabul edilmiş bir eksik (Bölüm 10).
- **Sayfalama yok.** `GetAllAsync` tüm kayıtları çekiyor. 200 çalışanda sorun değil, 50.000 çalışanda uygulamayı düşürür.

7. Gelmesi muhtemel sorular

SQL injection'a karşı ne yapıyorsunuz?

→ Tüm değerler **parametre** olarak geçiyor (`@Id` , `@EmployeeId`); hiçbir yerde kullanıcı girdisi SQL metnine string birleştirmeye eklenmiyor. Dapper parametreleri `SqlParameter` 'a çevirir, yani veri asla sorgu metninin parçası olmaz. Bu, ORM kullanmamanın en büyük riskiydi ve doğru şekilde kapatılmış durumda.

Migration olmadan ekipçe nasıl çalışırdınız?

→ `db/` klasörü şemanın tek doğruluk kaynağı ve git'te takip ediliyor. Her değişiklik yeni numaralı bir dosya; `05_full_setup.sql` ise sıfırdan kurulum için birleşik ve idempotent hâli. Bu, EF migration'ının **elle yapılan disiplinli hâli** — çalışır ama otomatik değil ve "veritabanı hangi sürümde" sorusunu kendiliğinden cevaplamaz. Ekip büyüseydi bir migration aracı (DbUp, FluentMigrator) eklemek gerekirdi.

Neden ilişkisel veritabanı? NoSQL olmaz mıydı?

→ Domain doğası gereği ilişkisel: departman→birim→çalışan→yönetici zinciri, izin talebi→onaylayan kişi, stajyer→mentor. Sorguların çoğu bu ilişkiler üzerinden birleştirme yapıyor ve tutarlılık (bir talep tam olarak bir çalışana veya bir stajyere ait olmalı) veritabanı kısıtıyla garanti ediliyor. NoSQL bu garantiyi uygulama koduna taşırdı.

İzin bakiyesini neden bir tabloda tutmuyorsun?

→ Bilinçli karar: **hesaplanabilen veri saklanmaz**. Hak, işe giriş tarihinden kıdem hesaplanarak bulunuyor; kullanılan gün ise onaylanmış + bekleyen taleplerden toplanıyor. Saklansaydı iki kayıt arasında tutarsızlık oluşabilirdi (klasik hata: bakiye tablosu ile talep tablosu birbirini tutmaz). Karşı tarafı da bilmek gerekir: hesaplamak her seferinde sorgu maliyeti demektir; çok büyük veri hacminde saklamak (ve tutarlılığı transaction'la korumak) tercih edilebilir.

İki Host: API + WebUI, Refit ve BaseResponse

Ana soru: MVC uygulaması doğrudan Application katmanını çağırırsa daha hızlı olmaz mıydı? Araya HTTP koymanın maliyeti neden göze alındı?

1. Ne yaptık

İki ayrı çalışan uygulama var ve aralarında **proje referansı yok**; yalnızca çalışma anında HTTP ile konuşuyorlar.

TARAYICI —cookie→ WebUI (MVC) —JWT/HTTP→ API → Application → SQL

WebUI'nin bildiği tek şey: Services/IXxxApi arayüzleri (Refit)

WebUI'nin bilmediği şey: Application, Domain, Infrastructure, veritabanı

2. Neden iki ayrı host

- İş mantığının tek sahibi olsun.** Yarın bir mobil uygulama ya da başka bir sistem entegrasyonu gelirse aynı API'yi tüketir. Kurallar ikinci kez yazılmaz.
- Gerçek dünyayı taklit eder.** Kurumsal projelerde backend ve frontend genellikle ayrı ekipler, ayrı deploy hatları. Bu ayrımı öğrenmek stajın hedeflerinden biri.
- Güvenlik sınırı netleşir.** API tek başına ayakta ve kendi başına korunuyor. WebUI'yi tamamen atlayıp API'ye doğrudan istek atsan da yetki kontrolleri aynen çalışır — çünkü otorite WebUI'de değil, API'de.

3. Refit — neden elle HttpClient yazmadık

WebUI'de API sözleşmesi sadece bir C# arayüzü olarak yazılıyor; gerisini Refit üretiyor:

```
public interface ILeaveRequestApi
{
    [Get("/api/leaverequests/{id}/detail")]
    Task<BaseResponse<LeaveDetailResponse>> GetDetailAsync(int id);

    [Post("/api/leaverequests/{id}/approve")]
    Task<BaseResponse<int?>> ApproveAsync(int id);
}
```

Elle yazılıysaydı aynı iki metot için tekrar eden ~15 satırlık kod gerekirdi:

```
var response = await _httpClient.GetAsync($"api/leaverequests/{id}/detail");
var json = await response.Content.ReadAsStringAsync();
var result = JsonSerializer.Deserialize<BaseResponse<LeaveDetailResponse>>(
    json, new JsonSerializerOptions { PropertyNameCaseInsensitive = true });
// ...status kontrolü, hata yönetimi, header ekleme – her metot için tekrar
```

Alternatif	Değerlendirme
Elle HttpClient	Bağımlılık yok, tam kontrol; ama her uç için tekrar eden serialize/status kodu ve daha çok hata yüzeyi.
RestSharp	Benzer iş; Refit'in <i>arayüzden üretim</i> yaklaşımı kadar tip güvenli değil.
OpenAPI'den istemci üretimi (NSwag / Kiota)	En güçlü alternatif. API'nin <code>swagger.json</code> 'ından istemci otomatik üretilirdi; aşağıdaki "elle senkron tutma" problemini kökünden çözerdi. Bedeli: build adımına kod üretimi eklemek ve üretilen kodu yönetmek.

4. Paylaşılan Contracts projesi neden yok

Aynı JSON'ın modeli iki yerde duruyor: `API/Models/...` ve `WebUI/Models/Api/...`. Ortak bir `Contracts` projesi yapıp iki tarafın da ona referans vermesi mümkündü — **bilinçli olarak yapılmadı**.

Ayrı modeller (mevcut karar)	Paylaşılan Contracts projesi
+ Gerçek gevşek bağıllık: iki taraf bağımsız sürümlenebilir + API'nin iç modeli değişince WebUI derlemesi kırılmaz + İstemci yalnızca ihtiyacı olan alanları modelleyebilir – JSON şeklini elle senkron tutmak gerekir – Bir alan adı değişirse hata derlemede değil, çalışma anında çıkar	+ Tek doğruluk kaynağı; alan adı değişimi derleme hatası verir + Daha az kod – İki uygulama aynı takvimde deploy edilmek zorunda kalır – Ortak proje zamanla "çöp çekmecesini" olur; istemciye ait kaygılar sunucu modeline sızar

Savunmada dürüst cümle: "Bu bir ödün. Gevşek bağıllığı seçtik, bedeli JSON şeklini iki tarafta elle senkron tutmak. Ölçek büyüseydi doğru çözüm Contracts projesi değil, OpenAPI şemasından istemci üretmek olurdu — hem tek kaynak hem gevşek bağıllık."

5. BaseResponse zarfı

```
public class BaseResponse<T>
{
    public bool IsSuccess { get; set; }
    public string? Message { get; set; }
    public T? Data { get; set; }
}
```

Başarı da hata da **aynı şekli** döndürüyor. İstemci tek tip deserialize ediyor: önce `IsSuccess` 'e bakıyor, doğruysa `Data` 'yı kullanıyor, değilse `Message` 'ı gösteriyor. Bu yüzden Refit'in exception fırlatma davranışı da kapatılmış — hata bir istisna değil, okunacak bir veri.

ZARFIN "DELİKSİZ" OLMASI — ÜÇ MEKANİZMA

Bir zarf ancak **istisnasız** uygulanırsa değerlidir. Üç ayrı kaçak yolu ayrı ayrı kapatılmış:

1. **GlobalExceptionHandler** — kod içinde patlayan her exception'ı zarfa çevirir.
2. **InvalidModelStateResponseFactory** — [ApiController] 'ın model binding hataları (bozuk JSON) exception üretmez, MVC kısa devre yapıp *ProblemDetails* döndürür. Fabrika değiştirilmeseydi bu yanıtlar zarfın dışında kalırdı.
3. **UseBaseResponseStatusCodes** — 401/403/404 gibi yanıtlar framework tarafından **gövdesiz** üretilir; bu middleware olmasaydı istemci boş gövde okuyup "bilinmeyen hata" derdi.

Bu üç maddeyi sayabilmek, "bir sözleşmeyi gerçekten uygulamak ile uygulamış gibi yapmak arasındaki farkı" anladığını gösterir — savunmada en çok puan getiren detaylardan biri.

6. Bedeli ve karşı görüş

BİLİNMESİ GEREKEN ELEŞTİRİ

Zarf deseninin yaygın eleştirisi şudur: "HTTP zaten başarı/başarısızlık bildiriyor; *IsSuccess* alanı bu bilgiyi tekrar ediyor. En kötü hâli ise her şeye 200 dönüp gövdede *IsSuccess=false* demektir — bu, HTTP'yi anlamsızlaştırır ve ara katmanların (proxy, cache, monitoring) hatayı görmesini engeller."

Bizim savunmamız: Bu projede status code **doğru** kalıyor — doğrulama hatası 400, bulunamadı 404, beklenmeyen 500. Zarf, status code'un yerine geçmiyor; yanına **insan tarafından okunabilir tek tip mesaj** ekliyor. Yani bilgi tekrarı var ama HTTP semantiği bozulmuyor.

Diğer maliyetler:

- **Ağ üzerinden çağrı, in-process çağrıdan yavaştır.** Her sayfa yüklemesi bir HTTP gidiş-dönüşü daha yapıyor. Bu ölçekte kabul edilebilir; sonuç sayfa açılışında milisaniyeler.
- **İki uygulama = iki deploy, iki konfigürasyon, iki log kaynağı.** İşletme yükü artar.
- **Hata ayıklama zorlaşır.** Bir sorunda WebUI mi API mi hatalı sorusunu ayırıştırmak gerekir.

7. Gelmesi muhtemel sorular

Neden SPA (React/Angular) veya Blazor değil de sunucuda render edilen MVC?

→ Üç sebep: (1) Ayrı bir JavaScript build zinciri, paket yönetimi ve state yönetimi karmaşası gelmiyor — proje odağı iş mantığında kalıyor. (2) Kimlik doğrulama sunucu tarafında cookie ile çok daha güvenli kurulabiliyor (token tarayıcıya hiç inmiyor — Bölüm 6). (3) Ekranlar form ağırlıklı ve gerçek zamanlı etkileşim gerektirmiyor; SPA'nın getirisi bu ekranlarda düşük kalırdı.

WebUI'de de rol kontrolü var; bu yetki kontrolü değil mi?

→ Hayır, **yalnızca kullanıcı deneyimi**. WebUI'de menü gizlemek ve buton saklamak kullanıcıyı yapamayacağı işe yönlendirmemek içindir. Gerçek yetki API'de: her uç `[Authorize]` ile korunuyor ve içerik ayrıca kişiye göre süzülüyor. WebUI tamamen atlansa bile güvenlik aynı kalır.

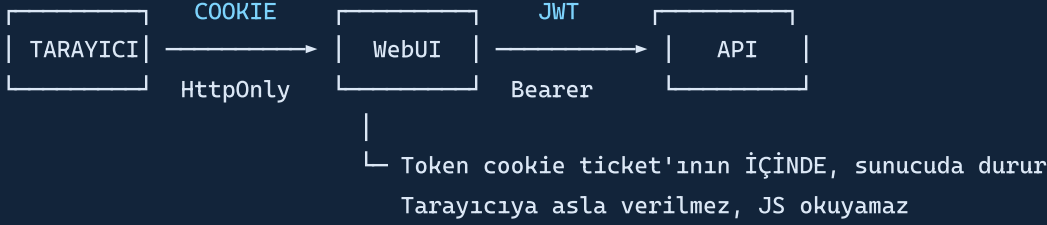
Neden CORS yapılandırması yok?

→ Gerekmiyor. API çağrıları **tarayıcıdan değil, WebUI sunucusundan** yapılıyor. CORS bir tarayıcı güvenlik mekanizmasıdır; sunucudan sunucuya isteklerde devreye girmez. Bu, mimarinin yan faydası: bir güvenlik yapılandırması daha az.

Kimlik Doğrulama: JWT + Cookie

Ana soru: Neden iki farklı mekanizma birden? Bir tanesi yetmez miydi?

1. Ne yaptık — iki güven sınırı, iki mekanizma



Akış şöyle işliyor:

1. Kullanıcı WebUI'de giriş formunu doldurur.
2. WebUI, API'nin login ucuna gider; API parolayı BCrypt ile doğrular ve **JWT üretir**.
3. WebUI bu token'ı **cookie ticket'ının içine** koyar ve tarayıcıya sadece HttpOnly cookie'yi yollar.
4. Sonraki her API çağrısında `BearerTokenHandler` token'ı ticket'tan okuyup `Authorization: Bearer ...` başlığına ekler.

```

protected override async Task<HttpResponseMessage> SendAsync(...)
{
    var token = await httpContext.GetTokenAsync("access_token");
    if (!string.IsNullOrEmpty(token))
        request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", token);

    return await base.SendAsync(request, cancellationTokens);
}
  
```

NEDEN İKİSİ BİRDEN — TEK CÜMLELİK CEVAP

Çünkü **iki farklı güven sınırı** var ve her birinin doğru aracı farklı. Tarayıcı güvenilir bir ortamdır: orada cookie kullanılır, çünkü HttpOnly ile JavaScript'ten korunabilir. API ise durum tutmayan (stateless) bir servistir: orada JWT kullanılır, çünkü her istek kendi kimliğini imzalı olarak taşır ve sunucunun oturum saklamasına gerek kalmaz.

2. Token neden localStorage'a konmadı

Bu, savunmada en sık sorulan güvenlik sorusudur ve cevabı nettir:

Yaklaşım	XSS durumunda ne olur
localStorage'da token	Sayfaya enjekte edilen tek satır JavaScript token'ı okuyup saldırganın sunucusuna gönderebilir. Hesap tamamen ele geçer ve token süresi dolana kadar iptal edilemez.
HttpOnly cookie (bizim seçim)	JavaScript cookie'yi okuyamaz . Saldırgan kullanıcı adına istek yaptırabilir ama kimlik bilgisini çalamaz — yani zarar oturumla sınırlı kalır, kalıcı olmaz.

Bizde durum bir kat daha iyi: token yalnızca cookie'de değil, **cookie'nin şifrelenmiş sunucu tarafı ticket'inin içinde**. Yani tarayıcıya inen değer token'ın kendisi bile değil.

Cookie'nin kendi riski CSRF'tir; bunun karşılığı ASP.NET Core MVC'nin form gönderimlerinde ürettiği antiforgery token'ı ve **SameSite=Lax** ayarıdır. "Cookie seçtim, dolayısıyla CSRF'i düşünmem gerekti" diyebilmek, konuyu ezberden değil anlayarak bildiğini gösterir.

3. "Kilitli doğan" uçlar — fallback policy

```
services.AddAuthorization(options =>
{
    options.FallbackPolicy = new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build();
});
```

Bu üç satır, üzerinde **[Authorize]** **yazmayan** her endpoint'i de kimlik doğrulaması ister hâle getiriyor. Açık kalması gereken uçlar (login) bilinçli olarak **[AllowAnonymous]** alıyor.

NEDEN BU, TERSİNDEN DAHA GÜVENLİ

Tersi yaklaşımda (her uca tek tek **[Authorize]** yazmak) bir gün biri yazmayı unutur. Unutulan uç **sessizce herkese açık kalır** — ne derleyici uyarır, ne test yakalar, çünkü uç "çalışıyor" görünür. Varsayılanı *kapalı* yapmak, unutmanın sonucunu "açıkta kalan uç" yerine "erişilemeyen uç"a çevirir. İlki sessiz bir güvenlik açığı, ikincisi hemen fark edilen bir hata. **Güvenlikte doğru varsayılan, gürültülü başarısızlıktır.**

4. JWT doğrulama parametreleri — her biri ne işe yarıyor

Parametre	Kapatılırsa ne olur
<code>ValidateIssuerSigningKey</code>	En kritik olanı. İmza doğrulanmazsa herkes kendine <code>Role: Admin</code> yazan bir token uydurabilir. Sistem tamamen çöker.
<code>ValidateIssuer</code> / <code>ValidateAudience</code>	Aynı imzalama anahtarını paylaşan başka bir sistemin token'ı burada da geçerli olur.
<code>ValidateLifetime</code>	Süresi dolmuş token sonsuza kadar geçerli kalır.
<code>ClockSkew = TimeSpan.Zero</code>	Varsayılan 5 dakikadır (sunucu saatleri kayabilir diye tolerans). Sıfırlanmasa "2 saatlik token" pratikte 2 saat 5 dakika yaşardı — öğrenme projesinde süreyi net görmek için sıfırlandı.

5. Parola: neden BCrypt

```
public string Hash(string password) => BCrypt.Net.BCrypt.HashPassword(password);  
public bool Verify(string p, string h) => BCrypt.Net.BCrypt.Verify(p, h);
```

Yaklaşım	Değerlendirme
Düz metin	Veritabanı sızarsa tüm parolalar açığa çıkar. Kabul edilemez.
MD5 / SHA-256	Bu algoritmalar hızlı olmak için tasarlandı — parola için tam olarak yanlış özellik. Modern bir GPU saniyede milyarlarca SHA-256 deneyebilir. Ayrıca salt yoksa rainbow table saldırısı çalışır.
BCrypt (bizim seçim)	Kasıtlı olarak yavaş ; <i>work factor</i> ile yavaşlık ayarlanabilir (donanım hızlandıkça artırılır). Salt'ı otomatik üretip hash'in içine gömer, yani her kullanıcının aynı parolası bile farklı hash üretir.
Argon2 / PBKDF2	Argon2 bugün teorik olarak en güçlüsü (belleğe de dirençli). BCrypt, .NET'te en olgun ve yaygın seçenek olduğu ve bu ölçekte fazlasıyla yeterli olduğu için tercih edildi.

6. Neden ASP.NET Core Identity kullanılmadı

Identity hazır kullanıcı/rol yönetimi, iki faktörlü doğrulama, hesap kilitleme, parola politikası getirirdi. İki sebeple alınmadı:

- **EF Core'a bağlı** gelir — Dapper kararımızla doğrudan çelişirdi (ya da Dapper için özel store yazmak gerekirdi, ki bu Identity'nin kolaylığını yok eder).
- **Öğrenme amacı.** Token üretimi, claim'ler, imza doğrulama, parola hash'leme hazır kutudan çıksaydı bu kavramların hiçbirini öğrenilmemiş olurdu.

Gerçek bir üretim projesinde tersi doğru olurdu: kimlik doğrulamayı elle yazmak, olgun bir kütüphaneyi kullanmaktan neredeyse her zaman risklidir. Bunu söyleyebilmek, kararın bağlamını anladığını gösterir.

7. Bedeli — bilinen zayıflıklar

BU ÜÇÜNÜ SORULMADAN SÖYLEMEK ÇOK DEĞERLİ

- **Refresh token yok.** Token 2 saat sonra ölüyor ve kullanıcı yeniden giriş yapmak zorunda. Refresh token olsaydı kısa ömürlü access token + uzun ömürlü refresh token ile hem güvenlik hem konfor artardı.
- **Token iptali yok.** JWT **stateless**'tir: sunucu onu "geri çağırılmaz". Çıkış yapıldığında yalnızca cookie siliniyor; token bir şekilde ele geçirilmişse süresi dolana kadar geçerli kalır. Çözümü: kara liste (blacklist) tutmak — ama bu JWT'nin durumsuzluk avantajını kısmen ortadan kaldırır. **Bu bir ödünleşimdir, hata değil.**
- **Rol değişikliği gecikmeli yansır.** Rol bilgisi token'ın içinde claim olarak taşınıyor. Bir kullanıcının rolü değiştirilirse, elindeki token dolana kadar eski rolüyle işlem yapabilir. Alternatif her istekte veritabanından rol okumaktır — ama o da JWT'yi anlamsızlaştırır.

8. Gelmesi muhtemel sorular

JWT'nin içinde ne var, güvenli mi?

→ Üç parça: header, payload (claim'ler — kullanıcı Id'si, kullanıcı adı, rol), imza. Payload **şifreli değil, sadece Base64** tür; herkes okuyabilir. Güvenliği sağlayan şey gizlilik değil, **imza**: içeriği değiştiren biri geçerli imza üretemez. Bu yüzden JWT'ye asla parola gibi gizli veri konmaz.

JWT secret'ını nerede tutuyorsun?

→ **Sadece API tarafında** ve `dotnet user-secrets` ile — koda ve git'e asla yazılmıyor. Yapılandırma bulunamazsa uygulama sessizce çalışmak yerine ne yapılması gerektiğini söyleyen bir hatayla açılmıyor. Üretimde bu değer environment variable ya da Azure Key Vault gibi bir sırt yöneticisinden gelir.

UseAuthentication ve UseAuthorization sırası neden önemli?

→ Authentication "sen kimsin?" sorusunu cevaplayıp `HttpContext.User` 'ı doldurur; Authorization "bu işe yetkin var mı?" sorusunu **o dolu kimlik üzerinden** cevaplar. Ters yazılırsa yetki kontrolü daima **boş kimlikle** çalışır ve her istek reddedilir.

Cookie ömrü neden 2 saat ve SlidingExpiration neden kapalı?

→ Cookie ömrü, API token'ının ömrüyle **aynı** olsun diye 2 saat. Sliding açık olsaydı cookie kullanıldıkça uzardı ama içindeki JWT uzamazdı — kullanıcı "giriş yapmış" görünüp API'den 401 almaya başlardı. İki mekanizmanın ömrünü eşitlemek bu tutarsızlığı engelliyor.

Hata Yönetimi ve Cross-Cutting Concerns

Ana soru: .NET'in RFC standardı `ProblemDetails` hazır gelirken neden kendi hata formatını yazdın?

1. Ne yaptık

Tüm işlenmemiş hatalar tek bir sınıfta yakalanıp aynı zarfa çevriliyor:

```
public sealed class GlobalExceptionHandler : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext ctx, Exception ex, ...)
    {
        var (status, message) = ex switch
        {
            FluentValidation.ValidationException v => // INPUT hatası
                (400, string.Join(" ", v.Errors.Select(e => e.ErrorMessage))),

            DataAnnotations.ValidationException b => // İŞ KURALI reddi
                (400, b.Message),

            _ => (500, "Beklenmeyen bir hata oluştu.") // geri kalan her şey
        };

        ctx.Response.StatusCode = status;
        await ctx.Response.WriteAsJsonAsync(BaseResponse<object>.Fail(message));
        return true;
    }
}
```

İKİ EXCEPTION TİPİNİN BİLİNÇLİ AYRIMI

Projede iki farklı `ValidationException` kullanılıyor ve bu kafa karışıklığı değil, **kasıtlı bir işaretleme**:

`FluentValidation.ValidationException` boru hattından gelen **input** hatasıdır (birden çok alan hatası taşıyabilir), `DataAnnotations.ValidationException` ise handler'daki **iş kuralı** reddidir. İkisi de 400 döner ama kaynağı tip düzeyinde ayırt edilebilir.

2. Neden ProblemDetails değil

`ProblemDetails` (RFC 7807), HTTP hata gövdeleri için standart bir formattır: `type`, `title`, `status`, `detail` alanları içerir. ASP.NET Core bunu hazır destekler.

ProblemDetails	BaseResponse zarfı (bizim seçim)
+ Endüstri standardı; üçüncü taraf istemciler tanır + Framework desteği hazır – Yalnızca hata yanıtlarını kapsar — başarı yanıtı farklı şekildedir – İstemcinin iki ayrı deserialize yolu tutması gerekir	+ Başarı ve hata aynı şekli döndürür → istemci tek yol + Refit ile birleşince exception yerine IsSuccess kontrolü – Standart değil; dış istemciler için dokümantasyon gerekir – HTTP status ile gövde bilgisini tekrar eder

Karar gerekçesi: bu API'nin **tek istemcisi kendi WebUI'miz**. Dış dünyaya açılan bir public API olsaydı ProblemDetails doğru seçim olurdu — çünkü orada standarda uymanın değeri, tek tip deserialize kolaylığından büyüktür.

3. Kullanmasaydık ne olurdu

Global handler olmasaydı, her controller metodunda şu blok tekrar ederdi:

```
try { ... }  
catch (ValidationException ex) { return BadRequest(BaseResponse<object>.Fail(ex.Message)); }  
catch (Exception) { return StatusCode(500, BaseResponse<object>.Fail("Hata")); }
```

Sonuçları:

- **Yüzlerce satır tekrar** ve her yeni uçta unutma riski.
- **Tutarsız mesajlar** — her controller kendi metnini uydurur.
- **Bilgi sızıntısı riski**. Bir yerde `ex.Message` doğrudan döndürülürse, istemci veritabanı tablo adlarını, dosya yollarını, bağlantı bilgilerini görebilir. Merkezî handler'da beklenmeyen hatalar bilinçli olarak **genel bir mesajla** dönüyor: *"Beklenmeyen bir hata oluştu."*

4. Ayrıntı: neden bazı mesajlar kasıtlı olarak genel

Model binding hatalarında istemciye *"İstek gövdesi okunamadı veya geçersiz."* dönüyor. Framework'ün ürettiği asıl mesaj şuna benzer: `"{'}' is an invalid start of a value. BytePositionInLine: 9"` — bu metin kullanıcıya hiçbir şey ifade etmez, ama sisteminin iç işleyişi hakkında bilgi verir. **Alan bazlı anlamlı doğrulama zaten FluentValidation'da yapılıyor**; buraya düşen hatalar teknik ayrıştırma hatalarıdır.

5. Bedeli

GERÇEK EKSİK: LOGLAMA

`GlobalExceptionHandler` hatayı zarfa çevirip **500 döndürüyor ama hiçbir yere loglamıyor**. Yani üretimde bir hata olduğunda geriye iz kalmaz — "kullanıcı hata aldı" dendiğinde neye bakılacağı belli değildir.

Doğrusu: handler'a `ILogger` enjekte edip 500'e düşen exception'ı tam yığın izi (stack trace) ile loglamak; ayrıca her isteğe bir korelasyon kimliği (correlation id) verip bu kimliği hem loga hem kullanıcıya gösterilen mesaja koymak. Böylece kullanıcı "şu kodu aldım" der, sen logda tam o isteği bulursun.

Bu eksiği **sorulmadan söylemek**, savunmada en iyi hamlelerden biridir: hem farkında olduğunu hem çözümünü bildiğini gösterir.

Diğer maliyetler:

- **Exception'la akış kontrolü.** İş kuralı reddi "istisnai" bir durum değildir; bunun için exception kullanmak tartışmalı bir tercihtir (Bölüm 3'teki Result tartışması).
- **Merkezî handler her şeyi 500'e indirger.** Farklı hata tipleri için özel davranış gerekirse (ör. 409 Conflict), `switch` ifadesi zamanla büyür.

6. Cross-cutting concern kavramı — toplu bakış

"Her yerde gereken ama hiçbir yerin asıl işi olmayan" endişelere cross-cutting concern denir. Projede nerede ele alındıklarını bilmek, mimariyi anladığını gösterir:

Endişe	Nerede çözülüyor	Mekanizma
Input doğrulama	Application	MediatR <code>ValidationBehavior</code>
Hata → HTTP yanıtı	API	<code>GlobalExceptionHandler</code>
Gövdesiz hata yanıtları	API	<code>UseBaseResponseStatusCodes</code> middleware
Kimlik doğrulama	API + WebUI	Middleware (JWT / Cookie)
Yetkilendirme varsayılanı	API + WebUI	Fallback policy / <code>AuthorizeFilter</code>
Token ekleme	WebUI	<code>BearerTokenHandler</code> (DelegatingHandler)
Kültür/tarih biçimi	WebUI	<code>UseRequestLocalization</code>
Loglama	— yok —	bilinen eksik
Transaction	— yok —	bilinen eksik

Bu tablonun güzelliđi řu: eksik olan iki satırın çözümü de **zaten var olan mekanizmalara bir halka eklemek** — loglama için bir **LoggingBehavior** , transaction için bir **TransactionBehavior** . Mimari, eksikleri kolay kapatılabilir hâle getirmiş durumda; bu da MediatR tercihinin gecikmeli bir kazancı.

Test Stratejisi

Ana soru: Neden sadece Application katmanı test ediliyor? Controller ve repository'ler neden test edilmiyor? Ve neden Moq gibi bir mock kütüphanesi yok?

1. Ne yaptık

`tests/HRManagement.Application.Tests` altında xUnit ile yazılmış testler var ve hepsi **tek bir katmanı** hedefliyor. Test edilen dört tür şey:

Tür	Örnek	Neden değerli
Handler'lar	<code>ApproveLeaveRequestCommandHandlerTests</code>	İş kurallarının kalbi — durum geçişleri burada doğrulanıyor
Paylaşılan kural sınıfları	<code>EmployeeVisibilityTests</code> , <code>MentorshipGuardTests</code>	"Kim kimi görebilir" gibi güvenlik etkili kurallar
Validator'lar	<code>CreateLeaveRequestCommandValidatorTests</code>	Input kurallarının izole doğrulaması
Saf hesaplama	<code>LeaveEntitlementTests</code>	Bağımlılığı olmayan, matematiksel kurallar — en ucuz ve en değerli testler

2. Neden Application katmanı

Test yazarken doğru soru "neyi test edeyim?" değil, "**hangi hata bana en pahalıya mal olur?**" dur. Cevap net: yanlış hesaplanmış bir izin bakiyesi ya da yanlış kişiye açılmış bir çalışan detayı. İkisi de Application katmanında.

- **Controller test etmek çoğunlukla framework'ü test etmektir.** Controller'lar bilinçli olarak ince: request'i command'a çevir, gönder, sonucu response'a çevir. Burada test edilecek mantık neredeyse yok.
- **Repository test etmek gerçek veritabanı ister.** Değerli ama pahalı; aşağıda ayrıca ele alınıyor.
- **Domain'de davranış yok** (Bölüm 9), yani test edilecek bir şey de yok.

3. Neden mock kütüphanesi yok — elle yazılmış Fake'ler

Projede `Moq` ya da `NSubstitute` yerine elle yazılmış sahte sınıflar var: `FakeLeaveRequestRepository` , `FakeUserRepository` , `FakeEmployeeRepository` .

Mock kütüphanesi (Moq/NSubstitute)

- + Tek satırda sahte nesne
- + Çağrı doğrulaması (`Verify`) hazır
- Testler kurulum koduyla dolar, okunurluğu düşer
- Ekstra bağımlılık (Moq'un geçmişteki *SponsorLink* tartışması, bir test kütüphanesinin bile tedarik zinciri riski taşıyabildiğini gösterdi)

Elle Fake sınıflar (bizim seçim)

- + Test kodu düz C#; okunurluğu yüksek
- + Fake'ler testler arasında yeniden kullanılıyor
- + Sahte davranış gerçekçi olabilir (ör. bellek içi sözlük)
- Her arayüz için elle sınıf yazmak gerekir
- Arayüze metot eklenince tüm fake'ler güncellenmeli

4. Testler dokümantasyon olarak

Test isimleri Türkçe ve cümle hâlinde yazılmış:

```
[Fact]
public async Task HR_calisaninin_talebi_yonetici_onayıyla_dogrudan_onaylanir()
{
    var (handler, repo) = CreateHandler(ownerRole: Role.HR);

    await handler.Handle(new ApproveLeaveRequestCommand(42, AdminUserId), default);

    Assert.Equal(LeaveStatus.Approved, repo.Updated!.Status); // İK aşaması atlandı
    Assert.Null(repo.Updated.HrApprovedByUserId);           // denetim izi: İK onayı YOK
}
```

Bunun değeri şu: **testler iş kurallarının yaşayan dokümantasyonudur**. Bir yıl sonra "İK çalışanının izni neden tek onayla geçiyor?" diye soran biri, dokümana değil bu test dosyasına bakar — ve doküman aksine, test **yalan söyleyemez**: kural değişirse test kırılır.

5. Bedeli — test piramidinin eksik katları

NEYİ TEST ETMİYORUZ

- **Repository SQL'leri hiç test edilmiyor.** Bu en büyük boşluk: yazım hatası olan bir SQL, kolon adı değişikliği ya da yanlış JOIN **ancak elle deneyerek** ortaya çıkar. Dapper kullandığımız için (SQL'i derleyici denetlemediği için) bu risk EF Core'a göre daha yüksek.
- **Uçtan uca akış test edilmiyor.** "Giriş yap → izin talebi oluştur → yönetici onaylasın" zinciri bir bütün olarak hiç doğrulanmıyor. Parçalar tek tek doğru olup birleşimleri hatalı olabilir.
- **Yetkilendirme testleri yok.** `[Authorize(Roles = "HR,Admin")]` etiketlerinin doğru uçlarda olduğunu doğrulayan bir test bulunmuyor. Bu etiketlerden birinin yanlışlıkla silinmesi sessiz bir güvenlik açığı yaratır.

Çözümü ne olurdu:

- **Integration test** — `WebApplicationFactory` ile API'yi bellek içinde ayağa kaldırıp gerçek HTTP istekleri atmak; yetki etiketlerini de bu şekilde test etmek.
- **Testcontainers** ya da LocalDB ile her test çalışmasında geçici bir SQL Server ayağa kaldırıp repository'leri gerçek veritabanına karşı doğrulamak.

6. Gelmesi muhtemel sorular

Test kapsamı (coverage) yüzde kaç?

→ Ölçülmedi — ve dürüst cevap şu: **yüzde tek başına anlamlı bir hedef değil**. %90 kapsam, kritik kuralların test edilmediği bir projede de elde edilebilir (getter/setter'ları test ederek). Buradaki strateji kapsamı değil **riski** hedefliyor: en pahalı hatalar (izin hakkı, görünürlük kuralları) test edilmiş durumda. Yine de kapsam ölçümü eklemek faydalı olurdu; proje `coverlet` paketini zaten içeriyor.

Unit test ile integration test farkı nedir?

→ **Unit test** tek bir birimi (sınıf/metot) izole eder; bağımlılıkları sahte nesnelerle değiştirir, hızlıdır (milisaniye), hata yerini kesin gösterir. **Integration test** birden çok bileşeni gerçek hâlleriyle birlikte çalıştırır (gerçek veritabanı, gerçek HTTP); yavaştır ama "parçalar birleşince çalışıyor mu" sorusunu cevaplar. Sağlıklı bir projede ikisi de bulunur — bu projede yalnızca ilki var.

TDD uyguladın mı?

→ Hayır; testler kural netleştikten sonra yazıldı. TDD'nin (önce test, sonra kod) faydası tasarımı test edilebilirliğe zorlamasıdır — ama bu proje mimari sayesinde zaten test edilebilir çıktı. Öğrenme aşamasında kuralı önce anlayıp sonra sabitlemek daha verimliydi.

Domain Modeli ve İş Kurallarının Yeri

Ana soru: Entity'lerin içinde hiç davranış yok, sadece property var. Bu "anemik domain model" değil mi — bir anti-pattern sayılmıyor mu?

1. Ne yaptık

Domain entity'leri saf veri taşıyıcıları:

```
public class Employee
{
    public int Id { get; set; }
    public string FirstName { get; set; } = string.Empty;
    public DateTime HireDate { get; set; }
    public int? ManagerId { get; set; } // null = en tepedeki kişi
    public int? AnnualLeaveDays { get; set; } // null = kıdemden hesapla
    public bool IsActive { get; set; } = true;
    ...
}
```

İş kuralları ise Application katmanında, iki yerde yaşıyor:

- **Handler'ın içinde** — o use-case'e özgü kurallar.
- **Paylaşılan kural sınıflarında** — birden çok use-case'in kullandığı kurallar: `LeaveApprovalGuard`, `EmployeeVisibility`, `MentorshipGuard`, `UnitManagerResolver`, `LeaveEntitlement`.

2. "Anemik model" eleştirisi ve cevabı

Domain-Driven Design savunucuları için ideal şudur: davranış, veriyle **aynı yerde** olmalı. Yani:

```
leaveRequest.Approve(approverId); // entity kendi durumunu korur
// yerine bizde:
handler.Handle(new ApproveLeaveRequestCommand(id, approverId));
```

Bu eleştiri teorik olarak haklı. Bizim gerekçelerimiz:

1. **Dapper doğrudan entity'ye eşliyor.** Dapper'ın çalışabilmesi için parametresiz kurucu ve yazılabilir property'ler gerekiyor. Zengin (rich) bir domain modelinde property'ler `private set` olurdu ve ayrı bir "persistence model" katmanı yazmak gerekirdi — proje ölçeğinde net bir kayıp.
2. **Kuralların çoğu tek bir entity'ye sığmıyor.** "Bu kişi bu izni onaylayabilir mi?" sorusu; talebi, talep sahibini, onaylayanın rolünü ve yönetici zincirini birlikte bilmeyi gerektiriyor. DDD'de bunun adı zaten **domain service**'tir — ve bizim `LeaveApprovalGuard` sınıfımız tam olarak odur. Yani kurallar dağınık değil; sadece entity'nin içinde değil.
3. **Kurallar test edilebilir durumda.** Anemik modelin asıl zararı kuralların controller'lara ve servislere dağılıp izlenemez hâle gelmesidir. Burada her kural adı olan bir sınıfta ve testi var — eleştirinin işaret ettiği **zarar oluşmamış**.

SAVUNMADA SÖYLENECEK OLGUN CÜMLE

"Evet, entity'lerimiz anemik. Bunu bilerek yaptık: Dapper'la zengin domain modeli ek bir eşleme katmanı gerektirdi. Anemik modelin asıl riski kuralların dağılmasıdır; biz o riski kuralları adlandırılmış, test edilmiş domain service sınıflarında toplayarak kapattık. Proje büyür ve kurallar tek entity etrafında yoğunlaşırsa, zengin modele geçmek Application katmanının dışını hiç etkilemez."

3. Enum kullanımı

Roller, izin türleri, izin durumları, kıdem seviyeleri enum olarak tanımlı ve veritabanında `int` olarak saklanıyor:

```
public enum Role { Admin = 1, HR = 2, Manager = 3, Employee = 4, Intern = 5 }
```

Enum (bizim seçim)	Lookup tablosu
+ Derleme zamanı güvenliği; yazım hatası imkânsız + Kod okunur: <code>LeaveStatus.PendingHr</code> + <code>switch</code> ifadelerinde eksik durum uyarısı alınır – Veritabanında anlamı yok — rapor yazan biri "2"nin ne olduğunu bilemez – Yeni değer eklemek kod değişikliği + deploy gerektirir	+ Veritabanı kendi kendini açıklar; JOIN ile isim gelir + Yeni değer eklemek veri işidir, deploy gerekmez – Kodda sihirli sayılar ya da ek sabitler oluşur – Her sorguya JOIN eklenir

Karar gerekçesi: bu değerler **iş mantığının parçası**, veri değil. Yeni bir izin durumu eklemek zaten kod değişikliği gerektirir (o durumda ne olacağını kod bilmeli). Enum, "bu liste kod tarafından bilinmek zorunda" gerçeğini dürüstçe ifade ediyor.

4. Projeye özgü tasarım kararları

a) Rol, yetkinin tamamı değil

Rol (`Admin` , `HR` , `Manager` ...) kaba bir kapıdır; asıl yetki **ilişkiden** gelir. Bir yönetici herkesin değil, yalnızca `ManagerId` zinciriyle kendisine bağlı çalışanların izinlerini görür; bir mentor yalnızca kendi stajyerini görür. Bu yüzden API'de rol etiketi (`[Authorize]`) **ile** Application'daki ilişki kontrolü (`EmployeeVisibility` , `MentorshipGuard`) birlikte çalışıyor.

Bu, savunmada öne çıkarılacak bir olgunluk göstergesi: çoğu öğrenci projesinde yetki yalnızca rol kontrolüdür ve "yönetici tüm çalışanları görür" gibi gerçek dışı bir modelle sonuçlanır.

b) Hesaplanabilen veri saklanmaz — ama istisnası var

Veri	Saklanıyor mu	Neden
Yıllık izin hakkı	Hayır	İşe giriş tarihinden hesaplanıyor. Saklansaydı kıdem ilerledikçe güncellenmesi gerekir, unutulursa tutarsızlık oluşurdu.
Kullanılan izin günü	Hayır	Onaylı + bekleyen taleplerden toplanıyor. Tek doğruluk kaynağı taleplerin kendisi.
Talebin iş günü sayısı	Evet	Çünkü girdileri (başlangıç/bitiş tarihi) talep oluşturulduktan sonra değişmiyor . Değişmeyen bir girdiden hesaplanan değeri saklamak tutarsızlık riski yaratmaz — sadece tekrar hesaplamayı önler.

Bu ayrımı açıklayabilmek, "denormalizasyon ne zaman doğrudur" sorusunun cevabını bildiğini gösterir: **girdileri değişebilen türetilmiş veri saklanmaz**.

c) Tarihler UTC saklanıyor

Veritabanında `SYSTUTCDATETIME()` kullanılıyor, gösterim anında yerel saate çevriliyor. Sebep: sunucunun saat dilimi değişirse ya da uygulama başka bir bölgeye taşınırsa, saklanmış karşılaştırmalar bozulmasın. Yerel saatle saklamak, yaz saati geçişlerinde tekrar eden/atlanan saatler yüzünden sinsi hatalar üretir.

d) `UpdatedAt` hiç güncellenmediyse `null` bırakılıyor

`CreatedAt` ile aynı değere set etmek "bu kayıt hiç değişmedi" bilgisini yok ederdi. Küçük ama düşünülmüş bir karar — denetim izi (audit trail) mantığının parçası.

5. Gelmesi muhtemel sorular

Domain katmanında hiç iş kuralı yoksa neden var?

→ Ortak dili (ubiquitous language) taşıyor: `Employee`, `LeaveRequest`, `SeniorityLevel`, `LeaveStatus` — sistemin kavram sözlüğü orada. Ayrıca hiçbir pakete bağımlı olmadığı için hem Application hem Infrastructure ona güvenle bağlanabiliyor. Bugün ince, ama **doğru yerde duran** bir katman: kurallar zenginleşirse taşınacağı yer belli.

Bir çalışanın yöneticisi kendisi olabilir mi? Döngü oluşabilir mi?

→ Bu, `ManagerId` gibi kendine referans veren alanların klasik riski ve veritabanı kısıtıyla tam olarak engellenemez. Projede yönetici atama kuralları Application'da denetleniyor (`ManagerAssignmentTests` bunu test ediyor). Bu tip yapısal kuralların **tek yerde ve testli** olması, tam olarak katmanlı mimariden beklenen fayda.

İzin talebi hem çalışana hem stajyere ait olabilir mi?

→ Hayır — `EmployeeId` veya `InternId` 'den **tam olarak biri** dolu olmak zorunda ve bunu veritabanı kısıtı (`CK_LeaveRequests_Requester`) garanti ediyor. Kritik bütünlük kurallarını yalnızca uygulama koduna bırakmayıp veritabanı seviyesinde de zorlamak bilinçli bir tercih: uygulamada bir hata olsa bile veri bozulamaz.

Zayıf Noktalar ve Bilinçli Ödünler

Savunmadaki en tehlikeli soru: "*Bu projenin zayıf yanı nedir?*" Cevabın "yok" olursa ya projeyi tanıımıyorsundur ya da dürüst değilsindir. Aşağıdaki tablo hazır cevabındır.

BU BÖLÜMÜ NASIL KULLANMALISIN

Eksikleri **sorulmadan** söylemek zayıflık değil, güç göstergesidir. Formül şu: "**Şu eksik — şu sebeple şimdilik kabul ettik — üretime çıkarken şöyle kapatılır.**" Bu üç parçalı cevap, bir eksiği bir mühendislik kararına dönüştürür.

1. Teknik borç tablosu

Eksik	Neden şimdilik kabul edildi	Üretimde çözümü
Transaction / Unit of Work yok	Her repository metodu kendi bağlantısını açıyor. Çok adımlı işlemlerde (hesap talebi onayı: kullanıcı oluştur + kaydı bağla) yarım kalmış durum oluşabilir.	MediatR'a bir <code>TransactionBehavior</code> eklemek; komutları tek bir <code>TransactionScope</code> içinde çalıştırmak.
Loglama yok	500 hataları hiçbir yere yazılmıyor; üretimde bir sorunun izini sürmek imkânsız olurdu.	<code>GlobalExceptionHandler</code> 'a <code>ILogger</code> ; ayrıca <code>LoggingBehavior</code> ile her komutun giriş/çıkışı ve korelasyon kimliği.
Refresh token yok	2 saat sonra yeniden giriş gerekiyor. Öğrenme projesinde kabul edilebilir sürtünme.	Kısa ömürlü access token + veritabanında saklanan, iptal edilebilir refresh token.
Sayfalama yok	Listeler tüm kayıtları çekiyor. Mevcut veri hacminde sorun çıkarmıyor.	<code>OFFSET / FETCH NEXT</code> ile sunucu tarafı sayfalama; query'lere <code>Page / PageSize</code> parametreleri.
Integration test yok	Birim testler en riskli kuralları kapsıyor; altyapı testi zaman almıştı.	<code>WebApplicationFactory</code> ile uçtan uca testler; repository'ler için <code>Testcontainers/LocalDB</code> .
SELECT * kullanımı	Yazması hızlı; şema değişimi henüz sorun çıkarmadı.	Kolonları açıkça listelemek — hem daha az veri hem şema değişikliğine dayanıklılık.
Eşzamanlılık kontrolü yok	İki kişi aynı talebi aynı anda onaylarsa son yazan kazanır (lost update).	<code>rowversion</code> kolonu ile iyimser kilitleme; <code>WHERE RowVersion = @expected</code> .
Rate limiting yok	İç ağda çalışan bir uygulama; kaba kuvvet riski düşük değerlendirildi.	ASP.NET Core'un yerleşik <code>RateLimiter</code> 'i, özellikle login ucunda.
Genel audit log yok	İzin onay izi (kim, ne zaman onayladı) tutuluyor ama "kim hangi kaydı değiştirdi" genel izi yok.	Değişiklik tablosu ya da temporal tables; alternatif olarak <code>AuditBehavior</code> .
DTO çoğaltması	Contracts projesi bilinçli olarak yok (Bölüm 5); bedeli elle senkron tutmak.	OpenAPI şemasından istemci kodu üretimi (NSwag/Kiota).
Önbellekleme (cache) yok	Departman/birim listeleri gibi neredeyse hiç değişmeyen veriler her istekte sorgulanıyor.	<code>IMemoryCache</code> ya da dağıtık cache; değişince invalidasyon.
Result yerine exception	Global handler tutarlı sonuç veriyor; akış okunur kalıyor.	<code>Result<T></code> tipi; hata yolunun derleyici tarafından denetlenmesi (Bölüm 3).

2. Bilinçli ödünler — "eksik" değil, "karar"

Aşağıdakiler yukarıdakilerden farklı: bunlar düzeltilecek borç değil, **bilerek verilmiş kararlar**. Aradaki farkı ifade edebilmek önemli.

Paylaşılan Contracts projesi yok

Kazanç: WebUI ve API bağımsız evrilebilir. Bedel: JSON şeklini iki tarafta elle senkron tutmak. *Gevşek bağlılık için ödenen bilinçli fiyat.*

EF Core yerine Dapper

Kazanç: SQL görünür, davranış öngörülebilir, öğrenme değeri yüksek. Bedel: migration ve change tracking elle. *Öğrenme projesinde doğru; büyük ekipli üründe tartışmalı.*

ProblemDetails yerine BaseResponse

Kazanç: başarı ve hata tek şekil, istemcide tek deserialize yolu. Bedel: standart dışı. *Tek istemcili API'de doğru; public API'de yanlış olurdu.*

ASP.NET Identity yerine elle auth

Kazanç: JWT, claim, hash kavramlarının gerçekten öğrenilmesi. Bedel: 2FA/lockout gibi hazır özelliklerin yokluğu. *Staj bağlamında doğru; üretimde olgun kütüphane tercih edilir.*

MediatR 12.5'te sabitleme

Sebepten teknik değil hukuki: 13+ ticari lisans. *Bağımlılık seçiminde lisans geleceğini de hesaba katmak gerektiğinin somut örneği.*

SPA yerine sunucuda render edilen MVC

Kazanç: ayrı JS build zinciri yok, token tarayıcıya hiç inmiyor. Bedel: zengin etkileşimli ekranlar için sınırlı. *Form ağırlıklı İK ekranlarına uygun.*

3. Kapanış: bu projeden öğrenilenler

- Mimari, kodun nereye yazılacağını söyleyen bir haritadır.** Değeri "güzel görünmek" değil, altı ay sonra bir kuralı nerede bulacağını bilmektir.
- Her teknoloji bir ödünleşimdir.** "En iyi" araç yoktur; bağlamına en uygun olan vardır. Dapper küçük ekipte ve öğrenme projesinde doğru, 50 kişilik ekipte muhtemelen değil.
- Güvenlikte doğru varsayılan kapalıdır.** Fallback policy örneği: unutmamanın sonucu "açıkta kalan uç" değil, "erişilemeyen uç" olmalı.
- Tekrarlanan her kural bir gün ayrışır.** Bu yüzden kurallar tek yerde: `LeaveEntitlement`, `EmployeeVisibility`, `MentorshipGuard`.
- Test edilemeyen mimari, mimari değildir.** Katman ayrımının en somut ispatı, iş kurallarının veritabanı olmadan test edilebilmesidir.

6. **Eksiđi bilmek, eksiđi olmamaktan daha deđerlidir.** Yukarıdaki tablo bu projenin en olgun parçası.

Hızlı Cevap Kartları

Savunmadan hemen önce son tekrar için. Her satır, o soruya verilecek ilk cümledir — detay gerekirse ilgili bölüme dönülür.

Neden Clean Architecture?

İş kuralları en uzun ömürlü varlıktır; onları teknolojiye bağımsız tutmak için. Kanıtı: iş kuralları veritabanı olmadan test edilebiliyor.

Katman kuralını ne koruyor?

Doküman değil, derleyici. WebUI'nin csproj'unda Application referansı yok — ihlal derleme hatası verir.

Neden MediatR?

Her use-case tek sınıf olsun ve validation/logging gibi ortak işler 26 handler'a tek tek yazılmak yerine tek boru hattında çalışsın diye.

MediatR neden 12.5'te sabit?

13+ sürüm ticari lisansa geçti. Teknik değil hukuki bir karar.

Bu tam CQRS mi?

Hayır. Ayrım kod düzeyinde; ayrı okuma veritabanı yok. Bu ölçekte eventual consistency'nin bedeli gereksizdi.

Validation'ı nereye koyarsın?

İkiye ayırırım: veritabanına bakmadan cevaplanabilen her şey Validator'da, sistemin durumuna bakan her şey Handler'da.

Neden FluentValidation?

Koşullu ("sadece hastalık izninde rapor zorunlu") ve çapraz alan ("bitiş $\geq$ başlangıç") kuralları attribute'larla ifade edilemez.

Neden Dapper, EF Core değil?

SQL görünür kalsın, üretilen sorgu sürpriz olmasın ve N+1 kazara oluşmasın diye. Bedeli: migration ve tekrar kod elle.

SQL injection'a karşı ne var?

Tüm değerler parametrelili geçiyor; hiçbir yerde kullanıcı girdisi SQL metnine birleştirilmiyor.

Repository neden var?

Dapper bir soyutlama sunmaz; repository olmasaydı SQL Application katmanına sızardı. Ayrıca testlerde fake'lenebiliyor.

Neden iki ayrı uygulama?

İş mantığının tek sahibi olsun (yarın mobil gelirse aynı API), ve güvenlik sınırı WebUI'de değil API'de olsun diye.

Neden Refit?

API sözleşmesi C# arayüzü olarak yazılıyor, istemci kodunu kütüphane üretiyor. Elle HttpClient/JSON kodu yazılmıyor.

Neden Contracts projesi yok?

Gerçek gevşek bağımlılık için. Bedeli JSON şeklini iki tarafta elle senkron tutmak; ölçek büyürse doğru çözüm OpenAPI'den istemci üretmek.

Neden kendi zarfın, ProblemDetails değil?

Başarı ve hata aynı şekli dönsün, istemci tek yol izlesin diye. Public API olsaydı ProblemDetails doğru olurdu.

Zarfın deliksiz olduğunu nasıl garanti ediyorsun?

Üç mekanizma: GlobalExceptionHandler, InvalidModelStateResponseFactory, UseBaseResponseStatusCodes.

Neden hem JWT hem cookie?

İki farklı güven sınırı var. Tarayıcı güvenilmez ortam → HttpOnly cookie. API stateless servis → imzalı JWT.

Token neden localStorage'da değil?

Orada olsaydı tek satır XSS ile çalınabilirdi. HttpOnly cookie'yi JavaScript okuyamaz; üstelik token cookie'nin sunucu tarafı ticket'ının içinde.

Neden CORS yok?

API çağrıları tarayıcıdan değil WebUI sunucusundan yapılıyor; CORS bir tarayıcı mekanizmasıdır.

Fallback policy ne işe yarıyor?

Uçlar kilitli doğuyor. Tersinde bir gün biri [Authorize] yazmayı unutur ve uç sessizce herkese açık kalır — bunu test de yakalamaz.

Neden BCrypt?

MD5/SHA hızlı olmak için tasarlandı; parola için tam ters özellik. BCrypt kasıtlı yavaş, salt'ı otomatik ve work factor ayarlanabilir.

JWT içeriđi gizli mi?

Hayır, payload sadece Base64 — herkes okur. Güvenliđi imza sağlar. Bu yüzden JWT'ye gizli veri konmaz.

Neden ASP.NET Identity yok?

EF Core'a bađlı geliyor (Dapper kararıyla çelişir) ve kavramların öğrenilmesi hedefti. Üretimde tersi doğru olurdu.

Neden sadece Application test ediliyor?

En pahalı hatalar orada. Controller test etmek çođunlukla framework'ü test etmektir.

Neden Moq yok?

Elle fake'ler test kodunu düz C# ve okunur tutuyor, bađımlılık eklemiyor. Bedeli her arayüz için sınıf yazmak.

Entity'ler neden anemik?

Dapper doğrudan entity'ye eşliyor. Kurallar dağılmadı; adlandırılmış, testli domain service sınıflarında toplandı.

İzin bakiyesi neden saklanmıyor?

Girdileri deđişebilen türetilmiş veri saklanmaz — saklansaydı iki kayıt arasında tutarsızlık oluşabilirdi.

Yetki sadece rol mü?

Hayır. Rol kaba kapı; asıl yetki ilişkiden gelir — yönetici kendi ekibini, mentor kendi stajyerini görür.

Projenin en büyük eksiđi ne?

Loglama ve transaction yönetimi. İkisi de mevcut mimariye birer MediatR behavior'ı ekleyerek kapatılabilir.

Terimler Sözlüğü

Savunmada geçmesi muhtemel kavramların net, kısa tanımları. Bir terimi kullanacaksan tanımını da verebilmelisin.

Clean Architecture

Bağımlılıkların dıştan içe doğru aktığı, iş kurallarının merkezde ve framework'lerden bağımsız olduğu katmanlı mimari yaklaşımı. Onion ve Hexagonal aynı ilkenin farklı adlandırmaları.

Dependency Inversion (DIP)

Üst seviye modüller alt seviye modüllere değil, ikisi de soyutlamalara bağlı olmalıdır. Pratikte: arayüzün sahibi onu kullanan taraftır. Bizde `IEmployeeRepository` Application'da tanımlı, Infrastructure'da uygulanır.

Dependency Injection (DI)

Bir sınıfın ihtiyaç duyduğu nesneleri kendisi oluşturmak yerine dışarıdan (genelde constructor'dan) alması. DIP'i hayata geçiren teknik.

Composition Root

Tüm bağımlılık eşlemelerinin yapıldığı tek nokta. Bu projede `API/Program.cs`.

CQRS

Command Query Responsibility Segregation — durumu değiştiren işlemler (Command) ile veri okuyan işlemlerin (Query) ayrılması. Tam hâli ayrı veritabanları kullanır; bu projede ayrım yalnızca kod düzeyindedir.

Mediator deseni

Nesnelerin birbirini doğrudan tanımak yerine bir aracı üzerinden haberleşmesi. Bağlantı sayısını $N \times N$ 'den N 'e düşürür.

Pipeline Behavior

Mediator'da bir mesajın handler'a ulaşmadan önce geçtiği ara halka. Cross-cutting concern'leri (validation, loglama, transaction) tek yerde uygulamayı sağlar.

Cross-cutting concern

Her yerde gereken ama hiçbir modülün asıl işi olmayan endişe: loglama, güvenlik, doğrulama, transaction.

DTO (Data Transfer Object)

Katmanlar veya sistemler arasında veri taşımak için kullanılan, davranışı olmayan nesne. Domain entity'sinin dış dünyaya sızmasını engeller.

Micro-ORM

SQL'i senin yazdığın, kütüphanenin yalnızca sonucu nesneye eşlediği veri erişim aracı (Dapper). Full ORM (EF Core) ise SQL'i kendisi üretir.

N+1 problemi

Bir liste sorgusundan sonra, listedeki her kayıt için ayrı bir sorgu daha atılması. 500 kayıt için 501 sorgu. ORM'lerde lazy loading ile kazara oluşur.

Idempotent

Birden çok kez çalıştırıldığında sonucu değişmeyen işlem. Projedeki SQL kurulum script'leri bu özelliktedir.

Unit of Work

Birden çok veri değişikliğini tek bir atomik işlem (transaction) olarak yönetme deseni. Bu projede yok — bilinen eksiklerden.

Anemic Domain Model

Yalnızca veri taşıyan, davranışı olmayan entity'ler. DDD açısından eleştirilir; asıl riski kuralların dağılmasıdır. Bu projede kurallar domain service sınıflarında toplanarak risk kapatılmıştır.

Domain Service

Tek bir entity'ye sığmayan, birden çok nesneyi ilgilendiren iş kuralını barındıran sınıf. Örnek: `LeaveApprovalGuard` , `EmployeeVisibility` .

Claim

Bir kimlik hakkında ileri sürülen bilgi parçası (kullanıcı Id'si, rolü). JWT'nin payload'ında taşınır ve imzayla korunur.

Stateless

Sunucunun istekler arasında oturum bilgisi tutmaması. JWT bunu sağlar; bedeli, üretilmiş bir token'ın geri çağırılmamasıdır.

HttpOnly cookie

JavaScript'in okuyamadığı cookie. XSS ile kimlik bilgisinin çalınmasını engeller.

XSS / CSRF

XSS: saldırganın sayfaya kendi JavaScript'ini çalıştırması. **CSRF:** kullanıcının oturumunu kullanarak farkında olmadan istek yaptırılması. İlkine karşı HttpOnly, ikincisine karşı antiforgery token ve SameSite kullanılır.

Salt / Work factor

Salt: her parolaya eklenen rastgele değer; aynı parolaların aynı hash üretmesini engeller. **Work factor:** hash'lemenin kaç tur süreceği; donanım hızlandıkça artırılır.

Fallback Policy

Üzerinde açık bir yetkilendirme kuralı bulunmayan endpoint'lere uygulanan varsayılan politika. Bu projede "kimlik doğrulaması zorunlu" olarak ayarlanmıştır.

İyimser kilitleme (Optimistic Concurrency)

Kaydı kilitlemek yerine, güncelleme anında verinin değişmediğini doğrulama (`rowversion`). Bu projede yok — bilinen eksiklerden.

Teknik borç

Bilerek ertelenmiş iyileştirme. Borç olması sorun değil; **farkında olunmaması** sorundur. Bölüm 10 bu projenin borç defteridir.